

Lectura 10 — IC-7602 Redes

Estudiante: Joshua Obando Castro | **Carné:** 2023130774

Fecha: 2026-05-06 | **Profesor:** Nereo Campos

1. ¿Qué métricas de rendimiento son importantes en Load Balancers y deben ser examinadas con Observabilidad?

Las métricas de rendimiento en un load balancer son críticas porque permiten identificar los límites de la implementación antes de que se conviertan en un problema real. El texto define tres métricas principales que hay que tener en cuenta, y cada una tiene un nivel de importancia diferente dependiendo del tipo de tráfico que maneje el sitio.

La primera y más importante en términos de rendimiento puro es **connections per second**, que indica cuántas conexiones entrantes puede aceptar el dispositivo en un segundo. También se le conoce como transacciones por segundo o sesiones por segundo dependiendo del vendor. Esta métrica es tan importante porque abrir y cerrar conexiones HTTP es una de las operaciones más costosas que puede hacer un stack de red, ya que implica completar el proceso del three-way handshake de TCP para cada conexión nueva, lo que involucra un paquete SYN del cliente al servidor, un ACK más SYN del servidor, y un ACK final del cliente. Todo ese proceso consume recursos del stack de red o del procesador de red del dispositivo, y es generalmente el primer punto donde cualquier dispositivo golpea una barrera de rendimiento antes de que se agoten los demás recursos. Si el sitio genera una cantidad alta de tráfico HTTP, esta es la métrica más importante que hay que monitorear en el load balancer.

La segunda métrica es **total concurrent connections**, que mide cuántas sesiones TCP abiertas puede mantener simultáneamente el dispositivo. Este número está limitado principalmente por la memoria disponible en el kernel o en el procesador de red del load balancer, y puede variar desde unos pocos miles hasta prácticamente infinito dependiendo del producto. En la práctica, antes de llegar al límite teórico de sesiones concurrentes, casi siempre se golpea antes otra barrera de rendimiento. Para tráfico UDP esto no aplica porque UDP es un protocolo sin conexión, entonces no hay concepto de sesión abierta que mantener.

La tercera métrica es **throughput**, medido en bits por segundo, que indica qué tan rápido puede el dispositivo pasar tráfico a través de su infraestructura interna. El throughput en realidad es el producto de dos variables: la tasa de transmisión de paquetes por segundo multiplicada por el tamaño de los payloads. Ethernet tiene un MTU (Maximum Transmittable Unit) de aproximadamente 1.5 KB, entonces un archivo más grande que eso se fragmenta en chunks de ese tamaño para el transporte. Lo que hace que el throughput sea relevante es que mientras más grande sea el payload, más eficiente es el uso del dispositivo porque se amortiza el overhead de cada paquete sobre más datos. Un HTTP GET de 100 bytes cabe en un solo paquete, mientras que una imagen de 32 KB requiere unos 21 paquetes, cada uno con su propio encabezado y overhead de procesamiento.

Una limitación histórica importante es **el muro de los 100 Mbps**. Muchos modelos de load balancer solo vienen con interfaces Fast Ethernet, lo que limita el throughput total a 100 Mbps. Las soluciones para esto son varias: usar Fast EtherChannel, que combina dos o más enlaces Fast Ethernet en uno solo; usar DSR (Direct Server Return), que elimina el tráfico de salida del dispositivo y reduce significativamente los

requerimientos de throughput; o migrar a Gigabit Ethernet (GigE) en el load balancer, que ya está bajando en costo y permite llegar a 1 Gbps con margen para crecer.

El comportamiento de estas métricas bajo carga describe un concepto importante llamado **the wall** o el muro. Cuando el tráfico crece, el tiempo de respuesta y el rendimiento se mantienen relativamente constantes hasta que se llega a cierto punto de saturación. A partir de ese punto la degradación es dramática: los tiempos de respuesta del HTTP se disparan, el tráfico se nivela y en casos extremos el dispositivo puede mostrar comportamientos impredecibles como reinicios, lockups o panic del kernel. Por eso, la observabilidad de estas métricas no es solo útil en producción sino que es esencial para planear la capacidad antes de llegar al muro.

Hay que tener en cuenta también que agregar features al load balancer tiene un costo en rendimiento. Un dispositivo que puede pasar 90 Mbps con solo Layer 4 activo puede degradar a 45 Mbps cuando se activa el parseo de URLs y la persistencia por cookies, porque eso implica inspeccionar mucho más del paquete, lo que es intensivo en CPU. En load balancers basados en hardware con chips ASIC esto puede ser más o menos predecible dependiendo de la arquitectura, pero en los basados en software la degradación es más lineal y proporcional al trabajo que se le agrega al procesador.

Las métricas también varían en importancia según el perfil de tráfico del sitio. Para tráfico **HTTP** la métrica más crítica es connections per second seguida por throughput, ya que HTTP 1.0 abría una conexión TCP por cada objeto de la página. Para tráfico **FTP o streaming** la métrica dominante es el throughput porque aunque las conexiones son pocas, la cantidad de datos transferidos por conexión es enorme, al punto de saturar fácilmente los 100 Mbps. Para **web stores** la métrica más importante son las conexiones sostenidas totales porque el objetivo es mantener a la mayor cantidad de clientes conectados simultáneamente, y este tipo de tráfico además requiere **stateful redundancy**, es decir que la información de sesión como cookies y datos de carrito de compra se replique entre los load balancers para que una falla en uno no cause que los usuarios pierdan su sesión.

2. Fuera de network hardware provisioning, ¿qué otra aplicación tiene esta información?

La información que proveen estas métricas va mucho más allá del dimensionamiento de hardware de red. El mismo razonamiento aplica directamente al diseño y la operación de cualquier sistema de software distribuido.

La métrica de **connections per second** es exactamente el concepto que se usa en los sistemas de bases de datos y APIs para definir los límites de concurrencia y los connection pools. Cuando se configura un pool de conexiones en una base de datos, lo que se está haciendo es controlar cuántas conexiones TCP nuevas se pueden abrir por segundo hacia el motor de base de datos, ya que abrir y cerrar conexiones tiene el mismo costo en ese contexto que en un load balancer. Si el pool es demasiado pequeño, las peticiones se encolan y el tiempo de respuesta sube. Si es demasiado grande, el motor de base de datos se satura antes de que el hardware sea el cuello de botella.

El concepto de **total concurrent connections** se traduce directamente en la capacidad de servidores de aplicación, microservicios y bases de datos. Cuando se hace capacity planning para un cluster de Kubernetes, por ejemplo, uno de los factores que determina cuántos pods se necesitan es cuántas conexiones simultáneas puede atender cada instancia del servicio antes de degradar. Esta es la misma métrica, aplicada al diseño de la capa de aplicación.

El **throughput** y el análisis del tamaño de payload se aplica al diseño de APIs y al análisis del costo de transferencia de datos en cloud providers. La misma lógica de que payloads más grandes son más eficientes por unidad de overhead es la que justifica el uso de protocolos como gRPC con Protocol Buffers en lugar de REST con JSON cuando el volumen de datos es alto, ya que los mensajes binarios son más compactos y reducen la cantidad de paquetes necesarios para transmitir la misma información.

El fenómeno del **muro** es directamente aplicable al diseño de sistemas resilientes. Toda la filosofía detrás de los circuit breakers, los rate limiters y los sistemas de backpressure en arquitecturas de microservicios existe precisamente para detectar cuándo un sistema se está acercando al muro y comenzar a degradar el servicio de forma controlada antes de que colapse de manera impredecible. Herramientas de observabilidad como Prometheus, Grafana y los dashboards de cloud providers están diseñadas para monitorear exactamente este tipo de métricas y alertar antes de que se llegue al punto de saturación.

Finalmente, el análisis de **perfiles de tráfico** que clasifica el comportamiento en HTTP, FTP/Streaming y Web Store tiene una aplicación directa en la definición de SLOs (Service Level Objectives) y en la estrategia de caché. Saber que el tráfico de una aplicación es principalmente HTTP con muchas conexiones cortas lleva a decisiones de diseño distintas que saber que es principalmente streaming con pocas conexiones largas y alto throughput, ya que cada perfil demanda una estrategia diferente tanto a nivel de infraestructura como a nivel de configuración de la capa de caché y de la base de datos.